

Arrays and Classes

Produced
by:

Dr. Siobhán Drohan,
Ms. Mairead Meagher,
Ms. Siobhán Roche.

Let's Look at arrays of different types

Arrays can store any type of data

Let's look at some examples:

1. Array of primitives - **int**
2. Array of objects – **String**
3. Array of objects - **Product**

An array can store any type of data.

Primitive Types

```
int numbers[] = new int[10];
```

```
byte smallNumbers[] = new byte[4];
```

```
char characters[] = new char[26];
```

Object Types

```
String[] words = new String[4];
```

```
Product products[] = new Product[10];
```

1) Array of **Primitives**
e.g. int

Structure of an **int** primitive array

int[] numbers;

numbers

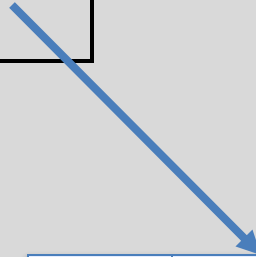
null

Structure of an **int** primitive array

```
int[] numbers;
```

```
numbers = new int[4];
```

numbers



0	0
1	0
2	0
3	0

Structure of an **int** primitive array

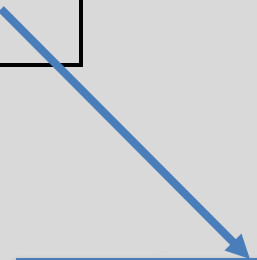
```
int[] numbers;
```

```
numbers = new int[4];
```

```
numbers[2] = 18;
```

We are directly
accessing the
element at index **2**
and setting it to a
value of **18**.

numbers



0	0
1	0
2	18
3	0

Structure of an **int** primitive array

```
int[] numbers;
```

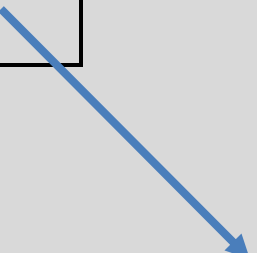
```
numbers = new int[4];
```

```
numbers[2] = 18;
```

```
numbers[0] = 12;
```

We are setting the
element at index **0**
to a value of **12**.

numbers



0	12
1	0
2	18
3	0

Structure of an **int** primitive array

```
int[] numbers;
```

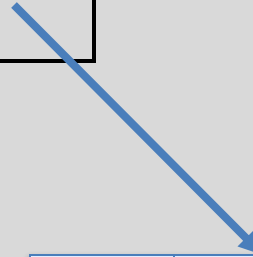
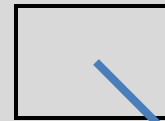
```
numbers = new int[4];
```

```
numbers[2] = 18;
```

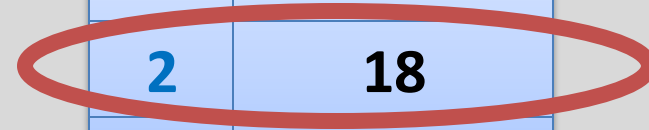
```
numbers[0] = 12;
```

```
print(numbers[2]);
```

numbers



0	12
1	0
2	18
3	0



Here we are printing the contents of
index location 2
i.e. 18 will be printed to the console.

2) Array of **Objects**
e.g. String

An array can store any type of data.

Primitive Types

```
int numbers[] = new int[10];
```

```
byte smallNumbers[] = new byte[4];
```

```
char characters[] = new char[26];
```

Object Types

```
String[] words = new String[4];
```



```
Product products[] = new Product[10];
```

Structure of a **String** object array

String[] words;

words

null

Structure of a **String** object array

```
String[] words;
```

```
words = new String[4];
```

words



0	null
1	null
2	null
3	null

Structure of a **String** object array

```
String[] words;
```

```
words = new String[4];
```

```
words[1] = "Dog";
```

words



0	null
1	
2	null
3	null



"Dog"

Structure of a **String** object array

```
String[] words;
```

```
words = new String[4];
```

```
words[1] = "Dog";
```

We are directly accessing the element at index **1** and setting it to a value of **"Dog"**.

words



0	null
1	
2	null
3	null



"Dog"

Structure of a **String** object array

```
String[] words;
```

```
words = new String[4];
```

```
words[1] = "Dog";
```

```
words[3] = "Cat";
```

words



0	null
1	
2	null
3	



"Dog"



"Cat"

Structure of a **String** object array

```
String[] words;
```

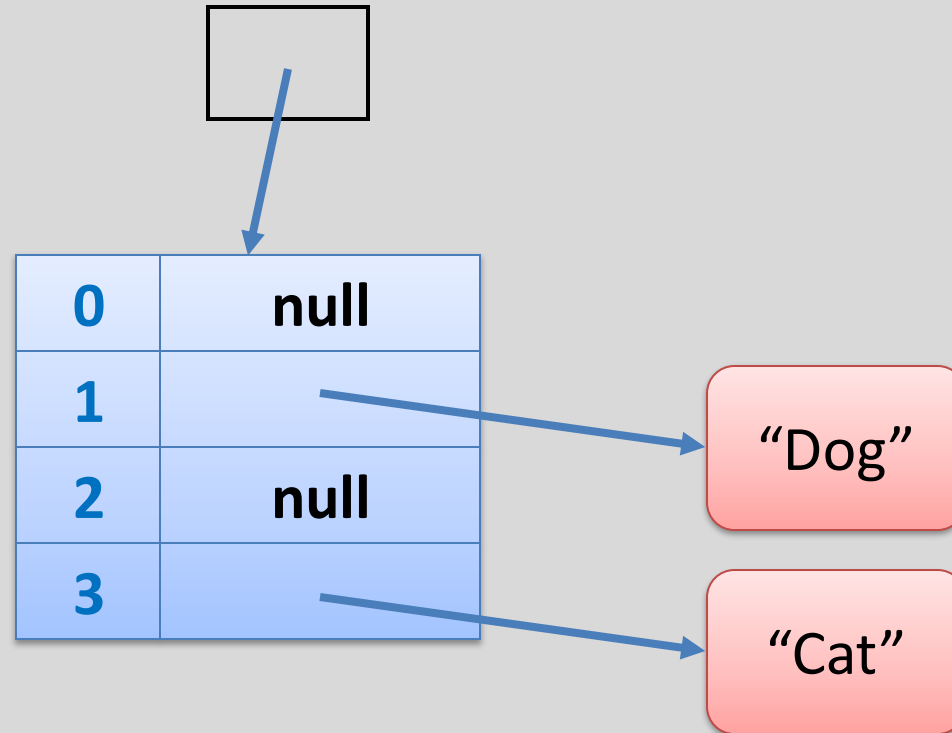
```
words = new String[4];
```

```
words[1] = "Dog";
```

```
words[3] = "Cat";
```

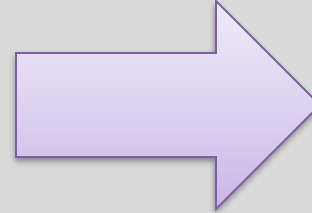
The element at index
3 is set to **"Cat"**.

words



Structure of a **String** object array

```
String words[];  
  
words = new String[4];  
  
words[1] = "Dog";  
words[3] = "Cat";  
  
for (int i=0; i < words.length; i++)  
{  
    System.out.println(words[i]);  
}
```



```
null  
Dog  
null  
Cat
```

3) Array of **Objects**
e.g. Product

An array can store any type of data.

Primitive Types

```
int numbers[] = new int[10];
```

```
byte smallNumbers[] = new byte[4];
```

```
char characters[] = new char[26];
```

Object Types

```
String[] words = new String[4];
```

```
Product products[] = new Product[10];
```



Product Class

**Object Type/
Class Name**

Methods
i.e. the behaviours of
the class

Fields
i.e. the attributes of
the class

```
Product
m Product(String, int, double, boolean)
m getProductName(): String
m getUnitCost(): double
m getProductCode(): int
m isInCurrentProductLine(): boolean
m setProductCode(int): void
m setProductName(String): void
m setUnitCost(double): void
m setIsCurrentProductLine(boolean): void
m toString(): String ↑Object
f productName: String = ""
f productCode: int = -1
f unitCost: double = 0
f isInCurrentProductLine: boolean = false
```

Structure of a **Product** primitive array

Product[] products;

products

null

Structure of a **Product** primitive array

```
Product[] products;
```

```
products = new Product[4];
```

products



0	null
1	null
2	null
3	null

Structure of a **Product** primitive array

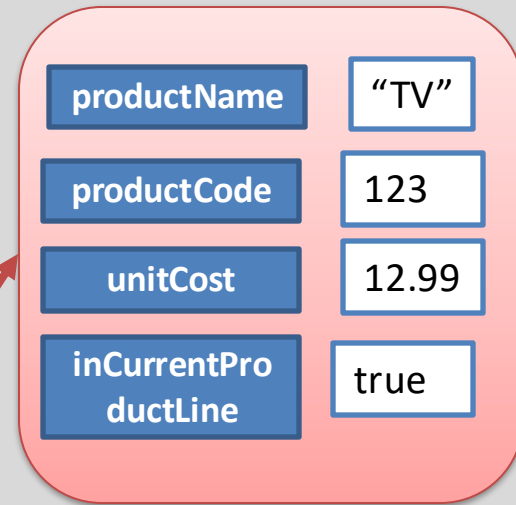
```
Product[] products;
```

```
products = new Product[4];
```

products



0	null
1	
2	null
3	null



```
products[1] = new Product("TV", 123, 12.99, true);
```


Example using a **Product** object array

```
public String listProducts() {  
  
    String listOfProducts = "";  
  
    for (int i = 0; i < total; i++) {  
        listOfProducts += i + ": " + products[i].toString() + "\n";  
    }  
  
    return listOfProducts;  
}  
}
```

Returns a String containing all the products stored in the primitive array.

Questions?

